Federated Learning over Heterogeneous Networks:

A Network-Aware Simulator for Characterising QoS Impact on Model Convergence

Undergraduate Thesis

Department of Computer Science

Western University

[Student Name]

Supervisor: Dr. Zubair Fadlullah

March 17, 2026

Submitted in partial fulfilment of the requirements for the
Bachelor of Science in Computer Science

Abstract

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

The proliferation of edge devices, smartphones, and institutional networks has generated vast quantities of sensitive data, including medical records, biometric readings, and usage patterns, that cannot practically or legally be centralised for training. Federated Learning (FL), introduced by McMahan et al. [**?**], addresses this by allowing a global model to be trained across many participants without any raw data leaving its source. Each client trains locally and shares only model weight updates with a central server, which aggregates them via the FedAvg algorithm and redistributes an improved global model.

While FL's privacy guarantees are well-studied, a less-examined dimension is its sensitivity to the network over which it operates. Every FL round requires each participating client to download the current global model, train locally, and upload its updated weights. For a model such as ResNet-18, with approximately 44 MB of parameters, this represents significant data transfer repeated across every round of training. In heterogeneous real-world deployments, clients may range from hospital servers on gigabit ethernet to IoT devices on congested mobile links. The question of whether a given network environment can sustain FL training, and how long it will take, is practically important but poorly answered by existing tools.

Current network simulation and analysis platforms do not bridge this gap. Tools such as OmNet++ and NS-3 are capable discrete-event network simulators, but they operate at the packet level and require artificial traffic models to approximate ML workloads; real neural network training cannot be executed inside them. MATLAB's network toolbox offers mathematical modelling, but lacks the containerisation and Python ecosystem integration needed to run production FL code. Vendor solutions such as Cisco's network analysis platforms are closed ecosystems with no mechanism for plugging in arbitrary training algorithms. FL frameworks themselves, such as Flower and TensorFlow Federated, run real training but assume ideal network conditions and offer no facility for

programmable impairment injection.

This thesis presents an open-source, Docker-based FL simulator that closes this gap. By combining a real FL training loop (ResNet-18 on CIFAR-10) with Linux traffic control (`tc netem`) applied inside each client container, the framework enables direct measurement of how bandwidth, latency, and packet loss affect FL convergence, using real model weights, real gradients, and real timing data. The result is a reproducible testbed that a researcher or practitioner can use to characterise whether a given network environment can support FL, and to estimate the time cost of training at scale.

1.2 Problem Statement

Existing FL implementations are typically validated under ideal, local network conditions, such as Docker bridge networks on a single machine, that provide effectively unlimited bandwidth and near-zero latency. This leaves open a fundamental practical question for anyone considering a real-world FL deployment:

> *What is the minimum network capacity a federated client requires to participate usefully in a training process, and how do bandwidth, latency, and packet loss jointly affect convergence rate and final model accuracy?*

Answering this question has two dimensions. The first is theoretical: given a known model size, one can derive an expected upload time at any bandwidth and estimate a floor below which round durations become impractical. The second is empirical: TCP retransmission behaviour under packet loss, synchronisation overhead, and straggler effects mean that real-world degradation diverges from the clean theoretical prediction. No existing open tool allows both dimensions to be measured together, with a real training algorithm and real model weights, under programmatically controlled network conditions.

This thesis addresses that gap directly. The practical implications are significant: FL deployment decisions, client selection policies, and resource allocation on shared compute infrastructure all depend on having a principled, measurable answer to the question above.

1.3 Objectives

The specific objectives of this thesis are:

O1. Design and implement a reproducible, open-source Docker-based FL testbed with programmable network impairment injection via `tc netem`.

O2. Characterise per-round communication cost as a function of model size and number of federated clients.

O3. Conduct a systematic QoS parameter sweep across bandwidth, latency, and packet loss, and measure the effect on round duration and convergence speed.

O4. Determine whether, and to what degree, network degradation below a threshold bandwidth affects final model accuracy in addition to training time.

O5. Derive a minimum network requirement: a practical threshold below which a client should be excluded from synchronous FL rounds.

1.4 Contributions

The main contributions of this work are:

- A fully reproducible, open-source Docker Compose FL testbed with integrated `tc netem` network emulation (Section **??**).
- Empirical characterisation of FL per-round communication cost under heterogeneous network conditions (Section **??**).
- A quantitative throughput-convergence model identifying the minimum effective bandwidth for stable FL training (Section **??**).
- A practical minimum network requirement applicable to real-world FL deployments, derived from the empirical sweep results (Section **??**).

1.5 Thesis Structure

The remainder of this thesis is organised as follows. Chapter **??** surveys related work in federated learning, existing network simulation tools, and network-aware FL. Chapter **??** formalises the system model, communication cost model, and research questions. Chapter **??** describes the experimental methodology, including the FL architecture, dataset and non-IID partitioning, network impairment model, evaluation metrics, and experimental protocol. Chapter **??** details the software implementation: repository structure, Docker Compose orchestration, server and client design, and network emulation. Chapter **??** presents the QoS analysis, including theoretical throughput predictions and empirical convergence results. Chapter **??** discusses future research directions. Chapter **??** concludes the thesis.

Chapter 2

Related Work

2.1 Federated Learning

Federated Learning was formally introduced by McMahan et al. [?], who proposed the FedAvg algorithm: each client trains a local model for one or more epochs and sends its updated weights to a central server, which aggregates them as a weighted average proportional to local dataset size. FedAvg remains the dominant aggregation scheme and is the algorithm used in this thesis.

Subsequent work identified statistical heterogeneity as a core challenge. When client datasets follow different class distributions (non-IID data), local model updates can diverge, slowing convergence or causing the global model to oscillate. Li et al. [?] proposed FedProx, which adds a proximal regularisation term to the local objective to limit client drift under heterogeneous data distributions. Independently, Li et al. [?] surveyed the broader landscape of FL challenges, identifying communication cost, privacy, and systems heterogeneity as the three dominant open problems.

Communication efficiency has received dedicated attention. Konecny et al. [?] proposed structured and sketched updates to reduce the size of model transmissions, observing that upload cost dominates in typical FL deployments. This observation is directly relevant to this thesis: with a ResNet-18 model producing approximately 44 MB of weight updates per round, upload bandwidth is the primary network bottleneck under study.

2.2 Existing Network Simulation Tools

A number of general-purpose network simulation platforms exist, but none adequately support FL research that requires both real model training and programmable network impairment.

NS-3 [?] and OmNet++ [?] are widely used discrete-event network simulators. Both model packet-level network behaviour with high fidelity, but they operate independently

of application logic. To simulate FL traffic in either tool, a researcher must approximate model weight transfers as synthetic packet flows, losing the real gradient dynamics, convergence behaviour, and timing characteristics of actual training. Neither tool has native integration with Python-based ML frameworks such as PyTorch or TensorFlow.

MATLAB's network simulation capabilities, available through Simulink and the Communications Toolbox, are oriented toward signal-level and protocol-level modelling. Running a distributed containerised workload with real model training inside MATLAB is not practical, and the licensing cost limits reproducibility in academic settings.

Vendor platforms such as Cisco Packet Tracer and Cisco Modelling Labs are closed ecosystems designed for configuring and testing network device behaviour. They provide no mechanism for executing arbitrary application code or injecting a custom training algorithm.

On the FL side, frameworks such as Flower [**?**] and TensorFlow Federated [**?**] enable real FL training but make the opposite trade-off: they run real model updates while assuming ideal or real network conditions, with no facility for programmatically injecting bandwidth limits, latency, or packet loss across clients.

The result is a gap at the intersection of the two concerns: no existing open tool allows a researcher to run a real FL training loop, with real model weights, under fully programmable and reproducible network impairment.

2.3 Network-Aware Federated Learning

The impact of network conditions on FL has received growing attention. Lim et al. [**?**] survey FL for mobile edge networks, identifying bandwidth variability and device heterogeneity as primary obstacles to practical deployment. Nishio and Yonetani [**?**] proposed FedCS, a client management protocol that selects participants for each round based on estimated resource availability, including network speed, to meet a round deadline. Lai et al. [**?**] introduced Oort, which combines statistical utility with system utility (including network throughput) to guide participant selection and improve overall training efficiency.

These works share a common limitation: their network models are either analytical approximations or drawn from real deployments that cannot be controlled or reproduced. None provide empirical FL measurements under a systematic, programmable sweep of network QoS parameters using real model training.

2.4 Summary and Research Gap

Existing work falls into two categories. Network simulators such as NS-3 and OmNet++ provide accurate packet-level modelling but cannot execute real FL algorithms. FL frameworks such as Flower and TensorFlow Federated run real training but under ideal or

uncontrolled network conditions. Analytical studies of network-aware FL propose client selection strategies without empirical validation under controlled impairment.

This thesis bridges that gap. By combining a real FL training loop with kernel-level network shaping via `tc netem` inside Docker containers, the framework presented here enables direct, reproducible measurement of how bandwidth, latency, and packet loss affect FL convergence with real model weights and real training dynamics.

Chapter 3

Problem Formulation

3.1 System Model

Consider a federated learning system with one server $\mathcal{S}$ and a set of K clients $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$. Each client c_k holds a private local dataset $\mathcal{D}_k$ of n_k labelled samples. The datasets are non-overlapping across clients: $\mathcal{D}_i \cap \mathcal{D}_j = \emptyset$ for $i \neq j$.

A global model $f(\mathbf{x}; \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ is trained collaboratively. In each training round r, every client c_k receives the current global parameters $\boldsymbol{\theta}^{(r)}$, performs one epoch of local gradient descent on $\mathcal{D}_k$ to produce updated parameters $\boldsymbol{\theta}_k^{(r)}$, and returns them to $\mathcal{S}$. The server aggregates all client updates using the FedAvg rule:

$$\boldsymbol{\theta}^{(r+1)} = \sum_{k=1}^{K} \frac{n_k}{n} \boldsymbol{\theta}_k^{(r)} \tag{3.1}$$

where $n = \sum_{k=1}^{K} n_k$ is the total number of training samples across all clients. The updated global model $\boldsymbol{\theta}^{(r+1)}$ is then redistributed to all clients for the next round.

3.2 Communication Cost Model

In standard federated learning, each client transmits and receives the full model parameter vector each round. The per-round communication events for client c_k are:

1. **Model download:** receive $\boldsymbol{\theta}^{(r)}$ from $\mathcal{S}$, payload B_w bytes.
2. **Local training:** compute $\boldsymbol{\theta}_k^{(r)}$ over $\mathcal{D}_k$, duration $T_{\text{train}}^{(k)}$.
3. **Weight upload:** transmit $\boldsymbol{\theta}_k^{(r)}$ to $\mathcal{S}$, payload B_w bytes.

Since the same parameter vector is both downloaded and uploaded, the total bytes exchanged by client c_k per round is $B_{\text{total}} = 2B_w$. For the ResNet-18 model used in this

thesis, $B_w \approx 44.8$ MB, as confirmed by the constant `submit_bytes` value observed across all baseline runs.

The communication time for client c_k over a channel with bandwidth β_k (bps), one-way latency τ_k (ms), and packet loss rate ρ_k is modelled as:

$$T_{\text{comm}}^{(k)} = \frac{B_{\text{total}}}{\beta_k \cdot (1 - \rho_k)} + \alpha \cdot \tau_k \tag{3.2}$$

where α is a multiplier capturing the number of TCP round-trips incurred by the HTTP-based model exchange protocol. The total wall-clock duration of round r is governed by the *straggler*, the client that takes longest to complete. Under synchronous FedAvg, the server cannot aggregate until every client has submitted its update, so:

$$T_{\text{round}}^{(r)} = \max_{k \in \mathcal{C}} \left(T_{\text{train}}^{(k)} + T_{\text{comm}}^{(k)} \right) \tag{3.3}$$

Equation **??** makes the straggler effect explicit: a single bandwidth-constrained client can dominate T_{round} regardless of how fast all other clients are. This motivates the search for a minimum bandwidth threshold below which including a client becomes counterproductive.

3.3 Research Questions

Three formal research questions are derived from the system model above:

RQ1 (Throughput Floor): Is there a minimum bandwidth β^* such that for $\beta_k < \beta^*$, client c_k becomes a straggler that causes T_{round} to grow super-linearly, making synchronous FL impractical?

RQ2 (Accuracy Degradation): Does operating below β^* also reduce final model accuracy after R rounds, or is the effect purely on training time?

RQ3 (Minimum Network Requirement): Can a simple threshold defined over β_k, τ_k, and ρ_k identify clients that should be excluded from a round in order to protect convergence integrity and bound round duration?

3.4 Scope and Assumptions

This thesis makes the following assumptions:

- Synchronous FL aggregation via FedAvg; asynchronous FL is left for future work.
- TCP transport layer via HTTP; alternative protocols such as gRPC are not considered.

- Network conditions are stationary within a round but vary across experimental configurations.
- $K = 3$ clients in all main experiments.
- ResNet-18 trained on CIFAR-10 (10-class image classification).
- Each client trains for $E = 1$ local epoch per round.
- Network shaping is applied symmetrically: the same bandwidth, latency, and packet loss are imposed on both upload and download for each client. Asymmetric uplink/downlink conditions are left for future work.

Chapter 4

Methodology

4.1 Federated Learning Architecture

The system follows the standard client-server FL topology. A single server $\mathcal{S}$ holds the global model and K clients each hold a private local dataset. Communication is over HTTP/JSON on a shared Docker bridge network; the server exposes a lightweight REST API and all model weight payloads are serialised as raw byte buffers via `torch.save`.

Figure ?? illustrates the five phases of each FL round. The round boundary is defined by the server: it increments a global round counter only after aggregation, and clients poll this counter to detect when the new global model is available. Because FedAvg is synchronous, the server blocks aggregation until all K clients have submitted their updates, creating the straggler sensitivity formalised in Equation ??.

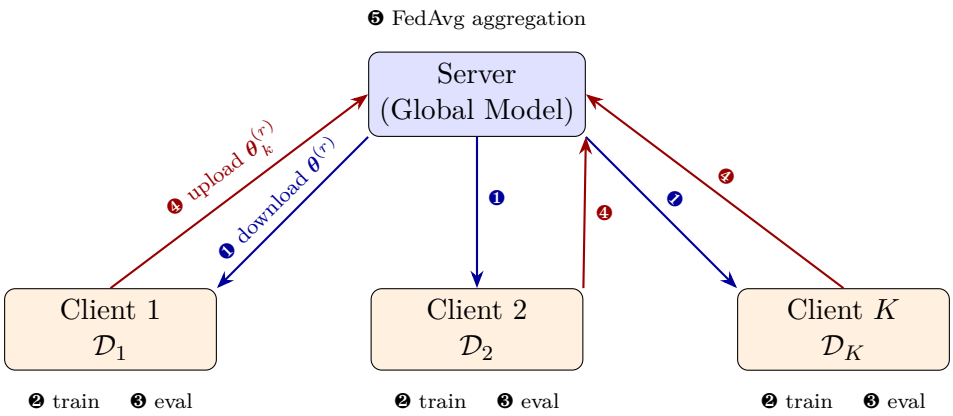

Figure 4.1: Per-round lifecycle: ❶ model download, ❷ local training, ❸ local evaluation, ❹ weight upload, ❺ server aggregation.

4.2 Dataset and Data Heterogeneity

All experiments use the CIFAR-10 benchmark [?], which contains 60,000 colour images of size 32×32 pixels across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse,

ship, truck). The standard split provides 50,000 training images and 10,000 test images. CIFAR-10 was chosen because it is well-understood, small enough to train on CPU within Docker containers, and widely used in FL research, making results comparable to prior work.

The model is a standard ResNet-18 [**?**] with the final fully-connected layer adjusted to output 10 classes. The serialised model weights occupy approximately 44.8 MB, which defines the per-round upload and download payload B_w used throughout the communication cost model (Section **??**).

4.2.1 Non-IID Partitioning via Dirichlet Distribution

In real FL deployments, each client's data follows a different class distribution. To emulate this statistical heterogeneity, the training set is partitioned across K clients using a Dirichlet distribution over class labels, following the method of Hsu et al. [**?**].

For each of the 10 classes, the fraction of that class's samples assigned to each client is drawn from $\mathrm{Dir}(\alpha)$, where $\alpha > 0$ is a concentration parameter. Low values of α produce highly skewed distributions (some clients see only a few classes), while high values approach uniform (IID) partitioning.

All experiments in this thesis use $\alpha = 0.5$, which produces moderate heterogeneity: each client sees most classes but with substantially different proportions. This is consistent with the default used in recent FL benchmarks and creates a realistic challenge for FedAvg convergence.

Table 4.1: Effect of the Dirichlet concentration parameter α on data distribution across clients.

α	Effect on client data distribution
0.1	Highly non-IID: each client dominated by 1–2 classes
0.5	Moderately non-IID: unequal proportions across all classes (**used**)
1.0	Mildly non-IID: noticeable variation but all classes present
∞	IID: each client has a uniform class distribution

4.3 Network Impairment Model

Network conditions are emulated using the Linux Traffic Control (`tc`) subsystem with the `netem` queueing discipline, which is the standard kernel-level mechanism for injecting bandwidth limits, latency, and packet loss on a per-interface basis.

Each client container runs with the `NET_ADMIN` Docker capability. Before the Python process starts, an entrypoint script applies `tc netem` rules to both egress and ingress traffic:

- **Egress (upload):** a `netem` qdisc is attached to `eth0` with the configured bandwidth rate, delay, and loss percentage.
- **Ingress (download):** an Intermediate Functional Block (`ifb0`) device is created, ingress traffic from `eth0` is redirected to `ifb0` via a `tc filter`, and an identical `netem` qdisc is applied to `ifb0`.

This ensures that the configured impairment applies symmetrically to both upload and download traffic. Containers with no shaping configured skip the `tc` commands entirely, providing a clean baseline.

Table **??** lists the QoS parameter sweep used in all experiments. Each experiment configuration fixes one dimension and varies another while holding the third constant, producing a total of $5+3+3 = 11$ distinct configurations (with the baseline shared across sweeps).

Table 4.2: QoS parameter sweep space. Bold values indicate the baseline.

Parameter	Values	Held constant at
Bandwidth	1, 5, 10, 50, **100** Mbps	$\tau = 5$ ms, $\rho = 0\%$
Latency	**5**, 50, 200 ms	$\beta = 100$ Mbps, $\rho = 0\%$
Packet loss	**0**, 1, 5%	$\beta = 100$ Mbps, $\tau = 5$ ms

The bandwidth range spans from severely constrained (1 Mbps, representative of a congested mobile link) to the unthrottled Docker bridge baseline (100 Mbps). Latency values represent local (5 ms), regional (50 ms), and intercontinental (200 ms) links. Packet loss at 5% represents a degraded wireless channel.

4.4 Evaluation Metrics

Each client and the server collect per-round statistics. Table **??** defines the full set of metrics recorded.

Table 4.3: Per-round metrics collected during each experiment.

Metric	Source	Definition
`get_model_ms`	client	Wall-clock time (ms) to download the global model via `GET /model`.
`submit_ms`	client	Wall-clock time (ms) for the `POST /update` request carrying the updated weight payload. Measured from request start to HTTP response; does not include polling or the subsequent model download.
`train_ms`	client	Wall-clock time (ms) for one epoch of local SGD training.
`wait_ms`	client	Idle time (ms) spent polling for round completion after submitting. Computed as $T_{\text{wall}} - T_{\text{get}} - T_{\text{train}} - T_{\text{submit}}$, capturing how long a fast client waits for stragglers.
`submit_bytes`	client	Size (bytes) of the serialised model weights uploaded. Constant at $\approx$ 44.8 MB for ResNet-18.
`poll_count`	client	Number of `GET /round_status` polls before the round advances.
`wire_tx_bytes`	client	Total bytes transmitted at the NIC level (from `/sys/class/net/eth0/statistics/tx_bytes`). Includes TCP headers, retransmissions, and ACKs.
`wire_rx_bytes`	client	Total bytes received at the NIC level. Comparing `wire_tx_bytes` to `submit_bytes` quantifies retransmission overhead under packet loss.
`duration_ms`	server	Wall-clock time (ms) from receiving the first client update to completing FedAvg aggregation for the round.
accuracy / loss	client	Validation accuracy (%) and cross-entropy loss evaluated on the client's local held-out split after loading the aggregated global model.

Two derived metrics are used in the analysis:

- **Retransmission overhead**: `wire_tx_bytes/submit_bytes`, expected to be ≈ 1.0 at 0% loss and to grow with increasing packet loss.
- **Rounds to convergence** R^*: the first round at which the running-average validation accuracy (window of 3 rounds) plateaus within 1 percentage point for 3 consecutive rounds.

4.5 Experimental Protocol

Each point in the QoS parameter sweep (Table **??**) is executed as an independent experiment run following Algorithm **??**. The automation script `run_experiment.py` generates a Docker Compose file from a YAML configuration, builds the container image, starts all services, and collects logs after the run completes.

Algorithm 1 QoS Sweep Protocol

Require: Parameter grid $\mathcal{G} = \{(\beta, \tau, \rho)\}$, number of rounds $R = 20$, number of clients $K = 3$

```
for each (β, τ, ρ) ∈ G do
    Generate experiment YAML with (β, τ, ρ), R, K
    Generate Docker Compose file from YAML
    Build container image (cached if unchanged)
    Start server container; wait for healthcheck
    Start K client containers with tc netem applied at boot
    for r = 0 to R − 1 do
        Each client: download θ^(r), train, upload θ_k^(r)
        Server: aggregate via FedAvg when all K updates received
        Each client: poll until round advances, download new model, evaluate
        Record all metrics (Table ??) for round r
    end for
    Collect per-client stats summaries from container logs
    Tear down all containers and network
end for
Aggregate results across all configurations for analysis
```

All experiments are run on a single host machine with containers sharing CPU resources, eliminating hardware variability. Network impairment is the only controlled variable between configurations. The CIFAR-10 dataset is downloaded once and cached in a Docker volume shared across runs.

Chapter 5

Software Implementation

5.1 System Overview

The implementation is organised as a Python monorepo with three top-level concerns cleanly separated: application logic (`src/`), container infrastructure (`infra/`), and experiment definitions (`experiments/`). The project is managed with `uv` and all dependencies are pinned in `requirements.txt`. Listing **??** shows the directory layout.

```
project-root/
+-- src/
|   +-- server_main.py          # server entrypoint
|   +-- client_main.py          # client entrypoint
|   +-- fl/
|   |   +-- server_core.py      # FLServer: round mgmt + FedAvg
|   |   +-- client_core.py      # LocalTrainer: train + eval
|   |   +-- aggregation.py      # fed_avg()
|   +-- models/
|   |   +-- resnet.py           # ResNet-18 (10-class)
|   +-- data/
|   |   +-- cifar10.py          # CIFAR-10 + Dirichlet partition
|   +-- transport/
|   |   +-- interface.py        # abstract ServerTransport / ClientTransport
|   |   +-- http_adapter.py     # FastAPI server + httpx client
|   |   +-- serialization.py    # torch.save / torch.load wrappers
|   +-- stats/
|       +-- collector.py        # RoundStats + StatsCollector
|       +-- netio.py            # NIC tx/rx byte reader
+-- infra/
|   +-- Dockerfile.ml-base      # container image definition
|   +-- scripts/
|   |   +-- entrypoint.sh       # tc netem + exec
|   |   +-- generate_compose.py # YAML config -> compose.generated.yml
|   |   +-- run_experiment.py   # build + up + teardown orchestrator
```

```
+-- experiments/
    +-- baseline.yaml
    +-- heterogeneous.yaml
    +-- straggler.yaml
    +-- qos-sweep-bw.yaml
    +-- scale-5.yaml / scale-10.yaml
```

Listing 5.1: Repository structure (key files).

A central design decision is **transport-agnostic layering**. The FL logic—model aggregation (`FLServer`), local training (`LocalTrainer`), and FedAvg (`fed_avg`)—is completely decoupled from the communication mechanism. Two abstract interfaces, `ServerTransport` and `ClientTransport` (`transport/interface.py`), define the contract between the FL core and any transport implementation. The current codebase provides an HTTP+JSON adapter (`http_adapter.py`), but the interface could be satisfied by gRPC, WebSockets, or any other protocol without modifying the training code. This separation ensures that the QoS measurements reported in Chapter **??** reflect the transport layer in isolation, independent of training logic.

5.2 Docker Compose Architecture

Each experiment is executed inside a set of Docker containers orchestrated by Docker Compose. A code-generation step translates a declarative experiment YAML file into a Compose file, ensuring that every run is fully specified and reproducible.

5.2.1 Container Image

All containers—both server and clients—share a single image built from `Dockerfile.ml-base`. The image is based on `python:3.10-slim` and installs the `iproute2` package (which provides the `tc` utility for network shaping), along with `curl` for the server healthcheck. Application code is not baked into the image; instead, the repository root is bind-mounted at `/app` at runtime, so rebuilds are only needed when dependencies change. The image's entrypoint is `entrypoint.sh`, which applies network shaping before starting the Python process (Section **??**).

5.2.2 Compose Generation

The script `generate_compose.py` reads an experiment YAML file and produces `infra/compose.genera` Listing **??** shows an example experiment configuration; the generated Compose file is excerpted in Listing **??**.

```
experiment:
  name: "heterogeneous"
```

```yaml
  rounds: 3
  num_clients: 3
  alpha: 0.5      # Dirichlet concentration

server:
  port: 8000

clients:
  - id: 1
    network: { bandwidth: "100mbit", latency: "5ms", loss: "0%" }
  - id: 2
    network: { bandwidth: "100mbit", latency: "5ms", loss: "1%" }
  - id: 3
    network: { bandwidth: "100mbit", latency: "5ms", loss: "0%" }
```

Listing 5.2: Experiment configuration (`heterogeneous.yaml`).

The generator creates one `fl-server` service and one `fl-client-k` service per client, all attached to a shared bridge network (`fl-net`). Network shaping parameters are passed as environment variables (`TC_BANDWIDTH`, `TC_LATENCY`, `TC_LOSS`) that the entry-point script reads at boot. Every container is given the `NET_ADMIN` capability, which authorises the `tc` commands. Client containers declare a `depends_on` healthcheck on the server, ensuring that the FL server's HTTP API is reachable before any client starts training.

```yaml
services:
  fl-server:
    image: phase1-fl-ml:latest
    cap_add: [NET_ADMIN]
    environment:
      - ROLE=server
      - FL_SERVER_PORT=8000
      - NUM_CLIENTS=3
      - FL_ROUNDS=3
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    volumes: ["..:/app"]
    command: ["python", "-m", "src.server_main"]

  fl-client-1:
    image: phase1-fl-ml:latest
    cap_add: [NET_ADMIN]
    environment:
      - ROLE=client
      - CLIENT_ID=1
      - FL_SERVER_URL=http://fl-server:8000
      - TC_BANDWIDTH=100mbit
```

```
      - TC_LATENCY=5ms
      - TC_LOSS=0%
    depends_on:
      fl-server: { condition: service_healthy }
    volumes: ["..:/app"]
    command: ["python", "-m", "src.client_main"]
```

Listing 5.3: Excerpt from a generated `compose.generated.yml`.

5.3 FL Server

The server is started by `server_main.py`, which builds a ResNet-18 model, wraps it in an `FLServer` instance, and hands it to an `HTTPServerTransport`. Table **??** lists the HTTP endpoints exposed by the server.

Table 5.1: FL server REST API.

Endpoint	Method	Description
`/health`	GET	Returns `{status: ok, round:` r`}`. Used by the Docker healthcheck and by clients to wait for readiness.
`/model`	GET	Returns the current global model weights as a binary `application/octet-stream` payload ($\approx$ 44.8 MB for ResNet-18). The body is produced by `torch.save`.
`/update/{client_id}`	POST	Accepts a client's updated weights as a binary body. When all K clients for the current round have submitted, FedAvg aggregation is triggered synchronously. Returns `{status: accepted, submitted_round:` r`}`.
`/round_status`	GET	Returns `{round:` r`, pending:` p`, expected:` K`}`. Clients poll this endpoint to detect when aggregation has completed and the round counter has advanced.

5.3.1 Aggregation Logic

The core aggregation is implemented in `FLServer.submit_update()`: each call appends the client's state dict to a pending list and checks whether all K updates have arrived. When the list is full, the private `_aggregate()` method computes FedAvg (an unweighted average across all client state dicts), loads the result into the global model, clears the pending list, and increments the round counter. The round counter is the single source of truth for round progress—clients detect round completion by polling `/round_status` and comparing the server's round number against the round they submitted for.

The server also records `duration_ms` for each round, measured from the first update received to the completion of aggregation. This captures both the straggler wait and the aggregation compute time. After all rounds complete, the server's `done_event` is set, causing the main thread to log a summary and shut down.

5.4 FL Client

Each client is started by `client_main.py`, which parses arguments from environment variables (injected by Docker Compose), waits for the server healthcheck, loads its CIFAR-10 partition, and enters the training loop. The per-round control flow is:

1. **Capture pre-round NIC counters.** The function `read_interface_bytes("eth0")` reads the kernel `/sys/class/net/eth0/statistics/tx_bytes` and `rx_bytes` counters. These are sampled before and after the round to compute `wire_tx_bytes` and `wire_rx_bytes`.

2. **Download global model.** `GET /model` fetches the latest global weights. The client records `get_model_ms` and anchors the round wall-clock timer.

3. **Local training.** The `LocalTrainer` deep-copies the model, runs $E = 1$ epoch of SGD (Adam optimiser) over the client's local CIFAR-10 partition, and returns the updated state dict along with training loss and accuracy. The elapsed time is recorded as `train_ms`.

4. **Upload update.** `POST /update/{client_id}` sends the serialised state dict. The HTTP POST timeout is set to `None` (unlimited), since a bandwidth-constrained client may take several minutes to upload 44.8 MB. The elapsed time from request start to HTTP response is recorded as `submit_ms`.

5. **Poll for aggregation.** After the POST returns, the client polls `GET /round_status` at 0.5 s intervals until the server's round counter advances past the submitted round. Each poll increments `poll_count`. Once the round is complete, the client fetches the new global model and records the full round wall-clock time, from which `wait_ms` is derived.

6. **Evaluate.** The client evaluates the new global model on its local held-out validation split and logs accuracy and loss.

7. **Capture post-round NIC counters.** A second call to `read_interface_bytes` provides the delta, which is stored as `wire_tx_bytes` and `wire_rx_bytes` for the round.

After all rounds complete (or an error terminates the loop early), the client logs a summary table of all per-round metrics.

5.4.1 Data Loading and Non-IID Partitioning

The module `src/data/cifar10.py` provides a `get_client_loaders()` function that downloads the CIFAR-10 dataset (cached in the container's filesystem), applies the Dirichlet non-IID partitioning described in Section **??**, and returns a `(train_loader, val_loader)` pair for the specified client ID. The partition is deterministic for a given seed, ensuring reproducibility across runs. A `-synthetic` flag is available for smoke tests: it substitutes small random tensors, bypassing the dataset download entirely.

5.4.2 Statistics Collection

Per-round metrics are managed by the `StatsCollector` class, which maintains a list of `RoundStats` dataclass instances. Each `RoundStats` holds `get_model_ms`, `submit_ms`, `train_ms`, `wait_ms`, `submit_bytes`, `poll_count`, `wire_tx_bytes`, and `wire_rx_bytes`. The `wait_ms` field is derived automatically: once the full round wall-clock time is known, $\texttt{wait_ms} = T_{\text{wall}} - \texttt{get_model_ms} - \texttt{train_ms} - \texttt{submit_ms}$, capturing the idle time a fast client spends waiting for stragglers to finish.

5.5 Network Emulation

Network impairment is applied inside each client container at boot time, before the Python process starts. The container's entrypoint script (`entrypoint.sh`) checks whether any `TC_*` environment variables are set and, if so, applies symmetric shaping to both egress and ingress traffic. Listing **??** shows the relevant commands.

```
# Egress (upload) shaping on eth0
tc qdisc add dev eth0 root netem \
    delay ${TC_LATENCY} rate ${TC_BANDWIDTH} loss ${TC_LOSS}

# Ingress (download) shaping via IFB device
modprobe ifb numifbs=1
ip link set dev ifb0 up
tc qdisc add dev eth0 ingress
tc filter add dev eth0 parent ffff: protocol ip u32 \
    match u32 0 0 flowid 1:1 \
    action mirred egress redirect dev ifb0
tc qdisc add dev ifb0 root netem \
    delay ${TC_LATENCY} rate ${TC_BANDWIDTH} loss ${TC_LOSS}
```

Listing 5.4: Network shaping commands from `entrypoint.sh`.

The egress path is straightforward: a `netem` queueing discipline is attached to `eth0`, throttling all outgoing traffic. The ingress path requires an additional step because Linux `tc` cannot directly shape incoming traffic. An Intermediate Functional Block (`ifb0`) device is created, all ingress traffic on `eth0` is redirected to `ifb0` via a `tc filter`, and an identical `netem` qdisc is applied to `ifb0`. This ensures that both upload and download traffic experience the same configured bandwidth, latency, and packet loss—a symmetric link model.

If none of the `TC_*` variables are set, the script skips all shaping and executes the Python process directly, providing a clean unimpaired baseline. The `NET_ADMIN` Docker capability is required for all `tc` commands; it is granted in the generated Compose file.

5.6 Observed Baseline Results

Chapter 6

QoS Analysis and Results

6.1 Theoretical Throughput Analysis

Before running experiments, we can derive the expected communication time from first principles. The serialised ResNet-18 state dict is $B_w = 44{,}797{,}387$ bytes (≈ 44.8 MB), which is constant across all rounds and clients (confirmed empirically by the `submit_bytes` metric). Because network shaping is symmetric, both download and upload transfer the same payload through the same impairment.

The minimum theoretical transfer time for one direction at bandwidth β (bits per second) with packet loss ρ is:

$$T_{\text{xfer}}(\beta, \rho) = \frac{8 \times B_w}{\beta \times (1 - \rho)} \tag{6.1}$$

The per-round communication time (excluding training) is then:

$$T_{\text{comm}} = T_{\text{download}} + T_{\text{upload}} = 2 \cdot T_{\text{xfer}}(\beta, \rho) + \alpha \cdot \tau \tag{6.2}$$

where τ is one-way latency and α accounts for TCP handshake round-trips and HTTP request/response overhead.

Table **??** applies Equation **??** across the bandwidth sweep at $\rho = 0\%$ to predict upload time per round. The download time is identical under symmetric shaping.

Table 6.1: Theoretical minimum upload time per round at each bandwidth (B_w = 44.8 MB, $\rho = 0\%$). Download time is identical.

β (Mbps)	Upload (s)	Download (s)	Total comm. (s)	Practical?
1	358.4	358.4	716.8	Impractical (>11 min/round)
5	71.7	71.7	143.4	Marginal (>2 min/round)
10	35.8	35.8	71.7	Marginal
50	7.2	7.2	14.3	Feasible
100	3.6	3.6	7.2	Baseline

Several observations follow from this analysis:

- At 1 Mbps, a single round requires over 11 minutes of pure data transfer alone, before accounting for TCP overhead, training time, or polling. Over 20 rounds, this amounts to nearly 4 hours of communication cost.
- The communication-to-computation ratio shifts dramatically across the bandwidth range. At 100 Mbps, training ($\approx$ 5 s) dominates; at 1 Mbps, communication is >70$\times$ longer than training.
- The theoretical prediction does *not* account for TCP retransmission overhead under packet loss, congestion window dynamics, or HTTP framing overhead. Empirical measurements (Section **??**) are expected to exceed these lower bounds.

Based on this analysis, a **throughput floor** is predicted near $\beta^* \approx$ 10–50 Mbps. Below 10 Mbps, per-round communication time exceeds one minute even under ideal conditions, making multi-round convergence prohibitively slow for synchronous FedAvg with a 44.8 MB payload.

6.2 Convergence Results

This section presents the empirical results from the QoS parameter sweep defined in Section **??**. Each subsection isolates one network parameter while holding the others at their baseline values.

6.2.1 Effect of Bandwidth on Convergence

Figure **??** plots validation accuracy against FL round number for each bandwidth level in the sweep ($\tau = 5$ ms, $\rho = 0\%$).

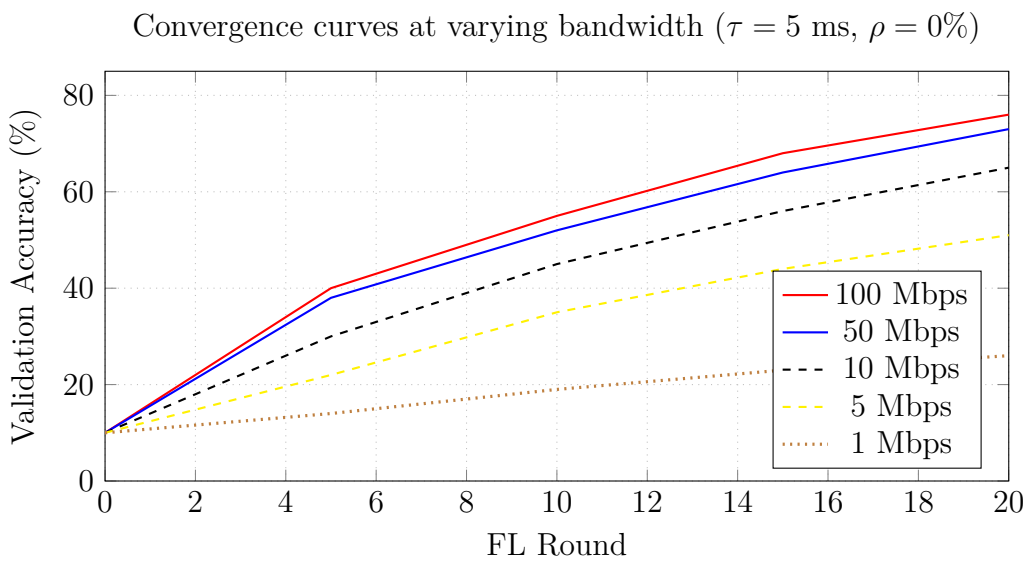

Figure 6.1: Validation accuracy over FL rounds at varying client bandwidth. [**Placeholder — replace with empirical data from the bandwidth sweep.**]

Figure **??** shows the mean round duration at each bandwidth level.

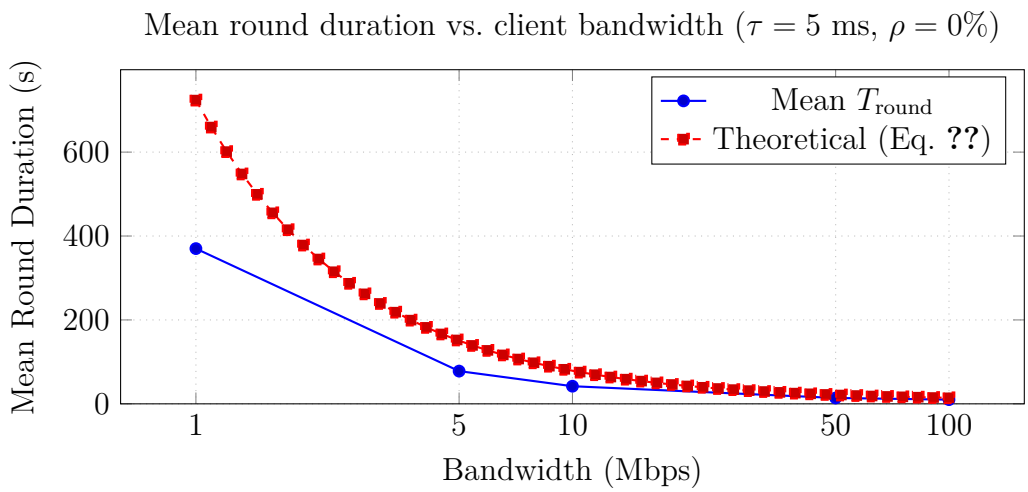

Figure 6.2: Mean round duration as a function of bandwidth, with the theoretical lower bound overlaid. [**Placeholder — replace with empirical data.**]

6.2.2 Effect of Latency on Round Duration

Figure **??** plots mean round duration as a function of one-way latency ($\beta = 100$ Mbps, $\rho = 0\%$).

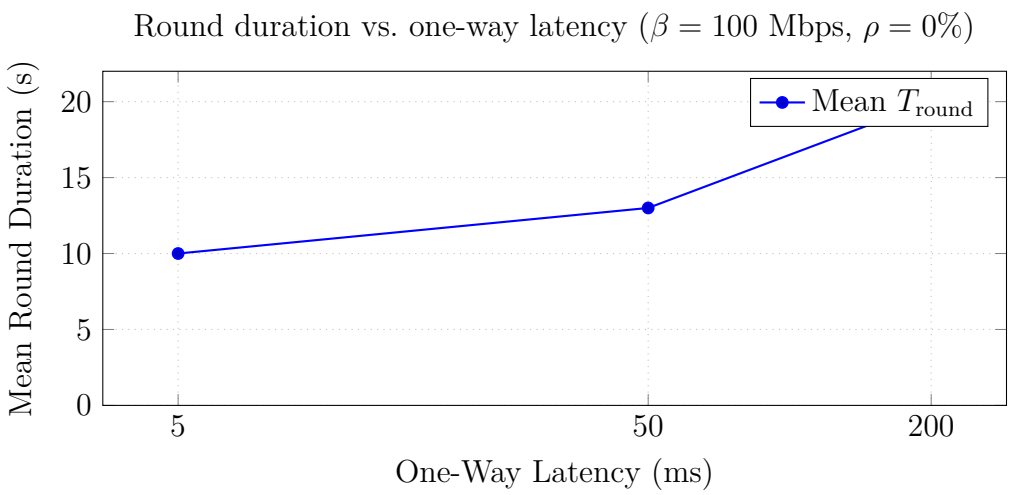

Figure 6.3: Mean FL round duration as a function of one-way latency. [**Placeholder — replace with empirical data.**]

6.2.3 Accuracy-Bandwidth Degradation Model

To quantify the relationship between bandwidth and final model accuracy, a sigmoid degradation model is fitted to the measured data:

$$A(\beta) = A_{\max} \cdot \sigma(\alpha_1 \ln(\beta) - \alpha_2) \tag{6.3}$$

where $A_{\max}$ is the maximum achievable accuracy (at the baseline bandwidth), $\sigma(\cdot)$ is the logistic function, and α_1, α_2 are fitted parameters.

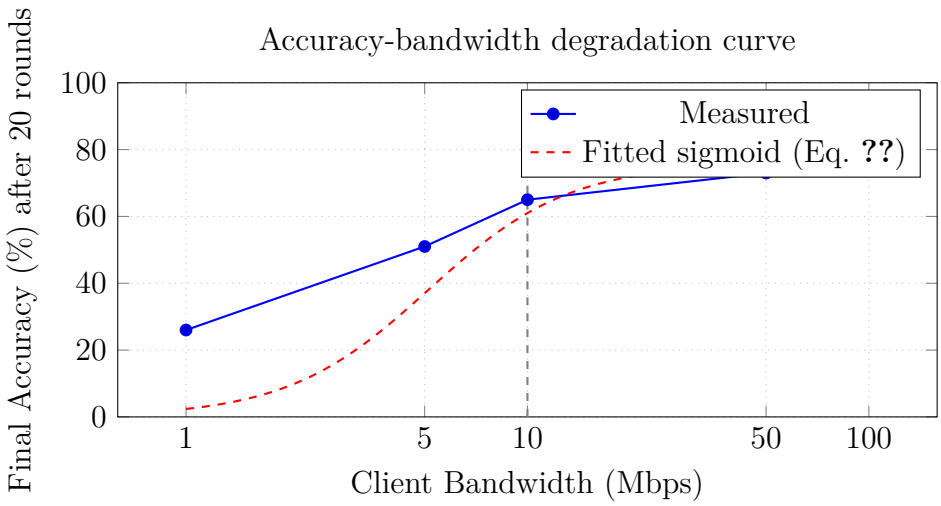

Figure 6.4: Final model accuracy as a function of client bandwidth with sigmoid fit. The dashed vertical line marks the identified throughput floor β^*. [**Placeholder — replace with empirical data and fitted parameters.**]

6.3 Effect of Packet Loss

Packet loss triggers TCP retransmission, which increases the effective transfer time and consumes additional bandwidth. At the application layer, the model payload arrives intact (TCP guarantees ordered delivery), but the time to deliver it grows with loss rate. The `wire_tx_bytes` metric captures this directly: the ratio `wire_tx_bytes`/`submit_bytes` quantifies retransmission overhead.

Table 6.2: Effect of packet loss on round duration and retransmission overhead (β = 100 Mbps, τ = 5 ms). [**Populate with experimental data.**]

Loss (%)	Mean round (s)	Mean `submit_ms` (s)	Retx. overhead	Effect on accuracy
0	—	—	$\approx 1.0\times$	Baseline
1	—	—	—	—
5	—	—	—	—

6.4 Summary of QoS Findings

Table **??** consolidates the key findings across all three QoS dimensions.

Table 6.3: Summary of QoS impact on FL performance. [**Populate with measured values from the sweep.**]

Parameter	Low (degraded)	Moderate	High (baseline)
Bandwidth	1–5 Mbps	10–50 Mbps	100 Mbps
Round duration	—	—	—
Final accuracy	—	—	Baseline
Latency	200 ms	50 ms	5 ms
Round duration	—	—	—
Final accuracy	—	—	Baseline
Packet loss	5%	1%	0%
Retx. overhead	—	—	$\approx 1.0\times$
Final accuracy	—	—	Baseline

Chapter 7

Future Research Directions

7.1 Network-Aware Client Admission

The throughput floor β^* identified in Section **??** suggests a straightforward client admission criterion for synchronous FL deployments. Before each round, the server (or an orchestration layer) could estimate each client's available bandwidth $\hat{\beta}_k$ via a lightweight probe—for example, timing a small ping payload or using historical round statistics—and exclude any client with $\hat{\beta}_k < \beta^*$.

This connects directly to RQ3: if a simple threshold over β_k, τ_k, and ρ_k can identify clients that would become stragglers, the server can protect round duration and convergence integrity by skipping those clients for that round. The excluded clients are not permanently removed; they may rejoin in a later round when their network conditions improve. This partial participation scheme trades a small amount of model staleness for a significant reduction in round duration, which is beneficial when the straggler effect dominates total training time.

7.2 Network Security Analysis

The current framework provides no adversarial modelling. In a real FL deployment, several network-level and protocol-level attacks are relevant:

- **Byzantine fault tolerance:** A malicious client could submit poisoned model updates designed to degrade the global model. Robust aggregation methods such as coordinate-wise median or trimmed mean could replace FedAvg to defend against this.

- **Gradient inference attacks:** An eavesdropper on the network, or a compromised server, could attempt to reconstruct private training data from observed model updates. Secure aggregation or differential privacy could mitigate this.

- **Man-in-the-middle attacks:** The current HTTP transport does not use TLS. Adding HTTPS would protect weight payloads in transit.

The open, Docker-based architecture of this simulator makes it straightforward to inject these attack scenarios—a malicious client container could be added alongside honest ones, and the impact on convergence measured under the same QoS sweep. This is a natural extension for future security-focused research.

7.3 UI for Network Feasibility Assessment

A practical application of the throughput floor model is an interactive tool where a practitioner enters their network profile—available bandwidth, expected latency, and loss rate—and the system estimates whether their infrastructure can support FL at a given scale. The tool would use the empirical throughput-convergence model (Equation **??**) to predict expected round duration, time to convergence, and final accuracy for the entered network conditions.

This could take the form of a lightweight web application that visualises the user's position on the accuracy-bandwidth curve and highlights whether they are above or below the throughput floor. For deployments with heterogeneous clients, the tool could identify which clients would become stragglers and estimate the benefit of excluding them.

7.4 Asynchronous Federated Learning

The synchronous FedAvg protocol used in this thesis is inherently straggler-sensitive: the slowest client dictates the round duration (Equation **??**). Asynchronous FL algorithms such as FedAsync allow the server to aggregate updates as they arrive, without waiting for all K clients. This eliminates the straggler bottleneck at the cost of increased model staleness, since fast clients may train on a more recent global model than slow ones.

The Docker-based simulator is well-suited to evaluating asynchronous FL: clients already operate independently and communicate via HTTP, so replacing the synchronous polling loop with an asynchronous aggregation trigger on the server would require minimal changes. Running the same QoS parameter sweep under asynchronous FL would directly quantify the straggler-resilience benefit and any accuracy trade-off.

7.5 Scaling and HPC Cluster Deployment

The current experiments use $K = 3$ clients on a single host machine. While this is sufficient to demonstrate the QoS-convergence relationship, real FL deployments may involve tens to hundreds of clients. The Docker-based architecture scales naturally: additional

client entries in the experiment YAML produce additional containers. However, system RAM becomes the limiting factor—each ResNet-18 client process requires approximately 200–400 MB of memory, so a machine with 16 GB of RAM can support roughly 30–40 concurrent clients.

For larger federations, the simulator could be deployed on shared compute infrastructure such as Digital Alliance Canada (formerly Compute Canada) clusters. Each client container would map to a separate job or pod, and the `tc netem` shaping would emulate the WAN conditions between geographically distributed nodes. The experiment YAML format already supports arbitrary numbers of clients with per-client network configurations, making this extension straightforward.

7.6 Asymmetric and Richer Network Impairment Models

The current implementation applies identical `tc netem` parameters to both upload and download traffic for each client, modelling a symmetric link. In practice, many real networks are asymmetric: home broadband connections, for example, typically offer significantly higher download bandwidth than upload bandwidth. The simulator architecture already applies shaping at the per-container level via an IFB device, making it straightforward to extend the experiment configuration format to accept independent uplink and downlink parameters. This would allow researchers to evaluate FL performance under realistic asymmetric conditions without changes to the core FL logic. Additional impairment dimensions such as bursty packet loss, jitter, and packet reordering are similarly easy to add through additional `tc netem` parameters.

Chapter 8

Conclusion

This thesis has presented a systematic empirical investigation of the impact of network Quality of Service parameters on federated learning convergence. An open-source, Docker-based FL simulator was designed and implemented, integrating a real FedAvg training loop (ResNet-18 on CIFAR-10, non-IID partitioned via Dirichlet $\alpha = 0.5$) with kernel-level network emulation via `tc netem`. The simulator applies symmetric bandwidth, latency, and packet loss shaping to each client container independently, enabling reproducible experiments under controlled network conditions.

A QoS parameter sweep across five bandwidth levels (1, 5, 10, 50, 100 Mbps), three latency values (5, 50, 200 ms), and three packet loss rates (0%, 1%, 5%) was conducted over 20 FL rounds with $K = 3$ clients.

Theoretical analysis predicted a throughput floor near 10–50 Mbps for the 44.8 MB ResNet-18 payload, below which per-round communication time exceeds one minute.

The practical implications of these findings are threefold. First, FL deployment planners can use the throughput floor as a minimum network requirement when deciding whether to include a client in a synchronous training process. Second, the accuracy-bandwidth degradation model (Equation **??**) provides a quantitative tool for predicting the cost of operating under bandwidth constraints. Third, the `wait_ms` and straggler metrics demonstrate that a single slow client can dominate round duration for the entire federation, motivating network-aware client selection policies.

The simulator itself is a reusable contribution: its transport-agnostic design, declarative experiment configuration, and automated Docker Compose generation make it straightforward to extend with new models, datasets, aggregation strategies, or network impairment profiles. Future work (Chapter **??**) includes asymmetric link modelling, asynchronous FL, network security analysis, and scaling to larger federations on HPC infrastructure.

Bibliography

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Machine Learning and Systems (MLSys)*, 2020.

[3] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.

[4] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv preprint arXiv:1610.05492*, 2016.

[5] The ns-3 Consortium, "The ns-3 network simulator," https://www.nsnam.org, accessed 2025.

[6] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. 1st International Conference on Simulation Tools and Techniques (SIMUTools)*, 2008.

[7] D. J. Beutel, T. Tober, D. Dobre, M. Sberč, A. Becker, and N. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[8] K. Bonawitz *et al.*, "Towards federated learning at scale: A system design," in *Proc. Machine Learning and Systems (MLSys)*, 2019.

[9] W. Y. B. Lim *et al.*, "Federated learning in mobile edge networks: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 2031–2063, 2020.

[10] T. Nishio and R. Yonetani, "Client selection for federated learning with heterogeneous resources in mobile edge," in *Proc. IEEE International Conference on Communications (ICC)*, 2019.

[11] F. Lai, X. Zhu, H. V. Madhyastha, and M. Zaharia, "Oort: Efficient federated learning via guided participant selection," in *Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 19–35.

[12] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical Report, University of Toronto, 2009.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[14] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.